

Specification of the Sonnet Programming Language

Version 0.1.0

StefanValentinT

Abstract

This specification details the syntactic form and evaluation of programs written in the Sonnet programming language. It details, how programs are represented, how these representations map to an understanding of terms, and how these terms get evaluated. In doing so it defines the requirements for a conforming implementation, thus establishing a single source of truth for the meaning of any program to promote the portability of the language itself.

CONTENTS

1. Introduction	1
1.1. Scope	1
1.2. Limitations	2
2. Conformance & Specification	2
3. Definitions	2
4. Syntax Specification	2
4.1. Grammar Rules	2
Glossary	3
References	3

1. INTRODUCTION

Sonnet ought to be beautiful in its simplicity, pristine in its architecture, bright-shining as the ivory tower, with a long spiral staircase winding its way all the way to the top, twisting round and round the tower. Sonnet shall be there for everyone, shall be easy to use because of its simplicity, but limitless in its reach. Sonnet shall bring back the simple, arcane joy of programming.

Therefore, the specification is written in a way to facilitate both unambiguity and accessibility for everyone. Whenever these goals conflicted, we opted to add important information using a NOTE, or, where needed, to provide additional reiteration or exemplification marked with an EXPLANATION, whilst not compromising the precise actual specification. Both shall be regarded as non-authoritative for the meaning of the language and may only aid in its understanding.

1.1. Scope

We first detail the actual representation of Sonnet programs, before specifying their syntactical structure. Thereafter the constraints and requirements enacted upon the meaning of a program are detailed and its evaluation via the reduction of terms. The means by which a program may interface with the

surrounding world are described and lastly a collection of definitions is given that a conforming implementation shall provide as minimal, opaque “atoms” of computation.

1.2. Limitations

No bound is given for the complexity of a program that may exceed the capabilities of a particular evaluator, compiler or processor. Although such limits seem to be necessary in practice, especially for data, we also do not demand minimal abilities of a system to not constrain the language by artificial limits.

The means by which a conforming evaluator is invoked or the way it evaluates a given program are left unspecified to allow for greater freedom of optimization. Only directly observable behaviour of the language itself shall be specified.

2. CONFORMANCE & SPECIFICATION

The use of “shall” denotes a mandatory requirement on an implementation, while “shall not” and “may not” declare a strict prohibition if used within a constraint

“May” is used to specify that some behaviour is conformant to the specification, noticeably it is not exclusive, signifying multiple interpretations may be within the bounds of conformance. “Need not” is used to explicitly denote the non-requiredness of a particular property.

A “conforming” implementation is any one that abides to all requirements laid out by this specification, including those requiring the rejection of invalid or erroneous programs. Similarly, a conforming program is one not rejected by a conforming implementation.

A implementation shall be accompanied by a clear definition of the behaviour in all cases of implementation-defined behaviour in this specification, for all cases where the characteristics do not match the recommendations of this specifications non-authorative text.

3. DEFINITIONS

The following definitions apply to the specification. Other terms are defined when they first occur or in a more appropriate section; when a term is defined it is being written in *italic* type.

- **Behaviour** is all action that influences the world and is therefore observable. Observable influence may also be called physical.
- **Bit** is a unit of data storage capable of holding one of two values, commonly referred to as $\{0, 1\}$.
- A **character** is a member of a set of characters that make up the textual representation of a Sonnet program.

4. SYNTAX SPECIFICATION

4.1. Grammar Rules

GLOSSARY

REFERENCES